

Convolutional Neural Networks

ECMM422

Fabrizio Costa

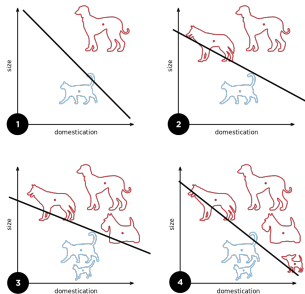
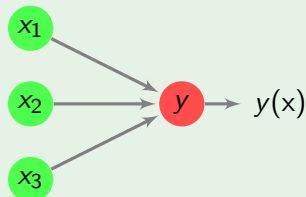
Winter 2026



A bit of history: The perceptron

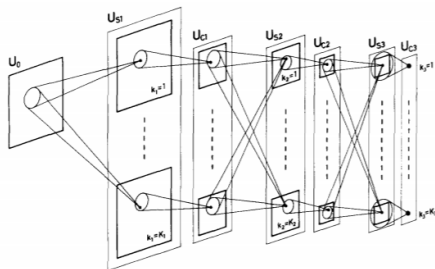
- ▶ Developed in hardware in the late 1950s
- ▶ Online training from gradient descent
- ▶ Guaranteed to converge to a solution, but not to the optimal one (optimal perceptron $\equiv$ SVM).

The Perceptron



A bit of history: Fukushima's Neocognitron

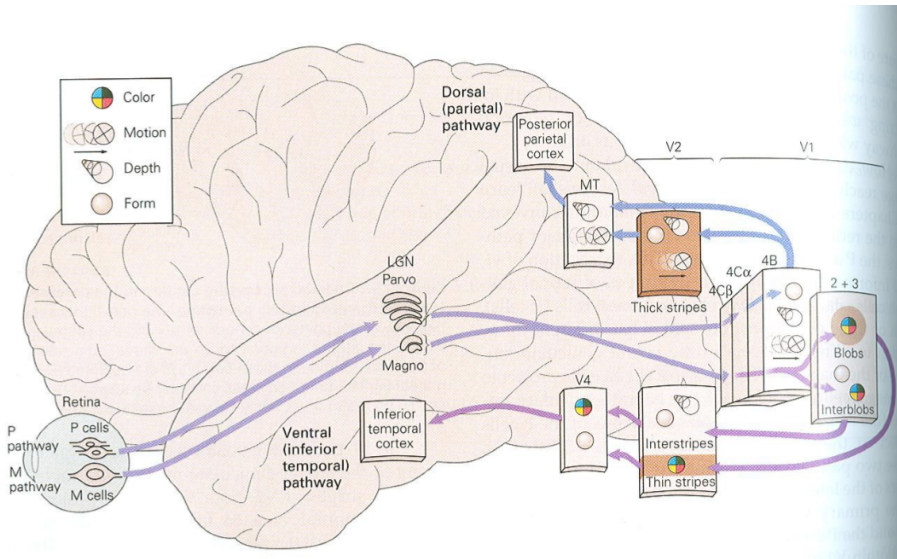
- ▶ Inspired by neuroscience, especially Hubel & Wiesel's discovery of simple and complex cells.
- ▶ “Grandmother cell” theory.
- ▶ Several layers of processing inspired from the brain
- ▶ Layers are not fully connected.



Fukushima, K. (1980) “Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position”, *Biological Cybernetics* 36, 193–202.

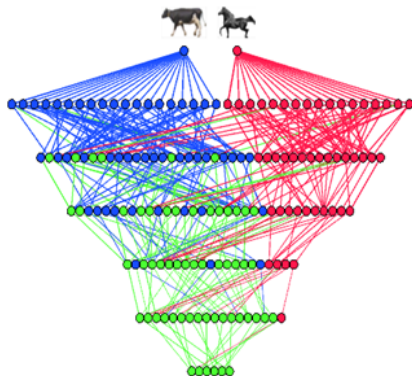
Motivation: Hierarchical learning

- ▶ **Problem:** Good features are hard to come by: complex to engineer, complex to learn from data.
- ▶ **Idea:** Learn good features by gradual abstraction, where each set of feature is abstracted from the previous set.
- ▶ In other words, we can define levels of abstraction as layers in a hierarchy, where the lower layers encode simpler features, and higher layers encode more abstract ones.
- ▶ Inspired by neuroscience evidence (especially, Hubel & Wiesel's seminal work on the visual system).



Motivation: Feature sharing

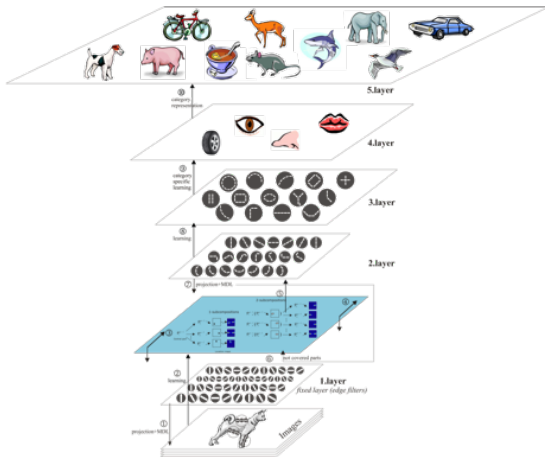
- ▶ With an SVM multiclass classification requires training N 1 vs all SVMs (for N classes), or $N(N-1)$ 1 vs. 1 SVMs.
- ▶ This is impractical for large number of classes
- ▶ But what is learnt differentiating a cow from a teacup can be generalised to a horse!
- ▶ In a hierarchy, features can allow sharing of the model's weights between classes, leading to sub-linear model growth with number of classes.



Fidler, S., M. Boben, and A. Leonardis (2009). "Evaluating multi-class learning strategies in a generative hierarchical framework for object detection." *Advances in Neural Information Processing (NIPS 2009)*.

Motivation: Feature sharing

- ▶ moreover, hierarchical learning can extract features at different **levels of abstraction**
- ▶ more abstract concepts are generally more invariant to local changes.
- ▶ higher predictive power, better generalisation.



Fidler, S., M. Boben, and A. Leonardis (2009). "Evaluating multi-class learning strategies in a generative hierarchical framework for object detection." *Advances in Neural Information Processing (NIPS 2009)*.

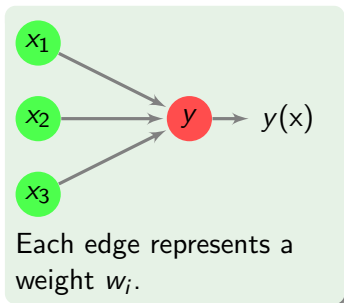
Motivation: Hierarchical learning

- ▶ What we need is **distributed** rules that can be applied **layer-wise**.
- ▶ This requires that the feature be **non-linear**, as a combination of linear transformations is a linear transformation (hence the same result could be obtained with a single layer).
- ▶ We will consider **feed-forward networks** where all features in a layer can be calculated from the feature response of the previous layer.
- ▶ **Question:** What learning algorithms can be used to learn such a hierarchy?
 - ▶ Using supervised learning where the features are optimised to improve classification or regression (eg, neural networks and backpropagation).
 - ▶ Using unsupervised learning where features are chosen to optimise a given criterion (e.g. reconstruction error with autoencoders).

Outline

- 1 Artificial neural networks
 - Artificial Neuron: The Perceptron
 - Learning synaptic weights: Gradient descent
- 2 Convolutional Neural Networks
 - LeNet and AlexNet
 - VGG16 and VGG19
 - Generalisation and transfer learning
 - Deep Residual Networks
- 3 Adversarial analysis

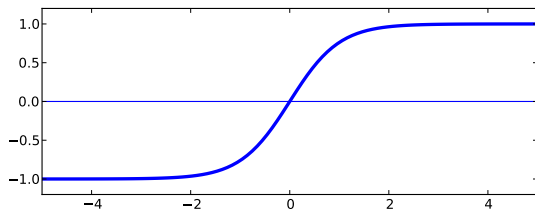
Artificial neuron



The prediction function is given by:

$$y(x) = \phi \left(b + \sum_i w_i x_i \right) \quad (1)$$

Each hidden layer node sums its weighted input and passes it through a *sigmoidal nonlinearity* ϕ

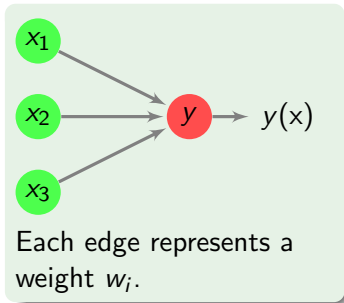


For example:

$$\phi_t(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2)$$

$$\phi_s(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

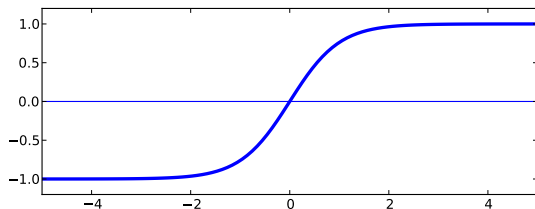
Artificial neuron



The prediction function is given by:

$$y(x) = \phi(Wx + b) \quad (1)$$

Each hidden layer node sums its weighted input and passes it through a *sigmoidal nonlinearity* ϕ

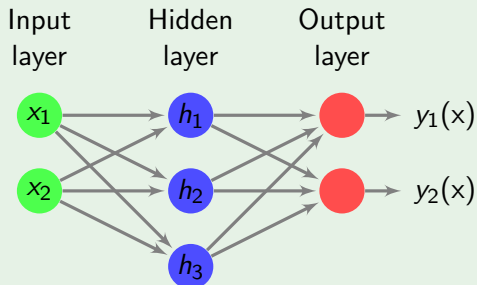


For example:

$$\phi_t(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2)$$

$$\phi_s(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

Artificial neuron network (aka, multilayer perceptron)

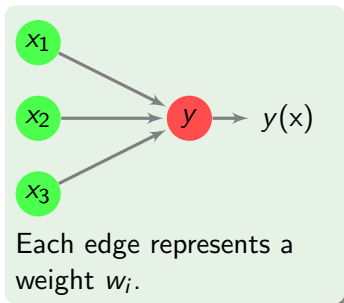


The parameters to learn are $\theta = (b^1, W^1, b^2, W^2)$.

The output units are given by

$$y_k(x) = \phi \left(b_k^2 + \sum_j W_{jk}^2 \phi \left(b_j^1 + \sum_i W_{ij}^1 x_i \right) \right) \quad (4)$$

Learning synaptic weights: Perceptron

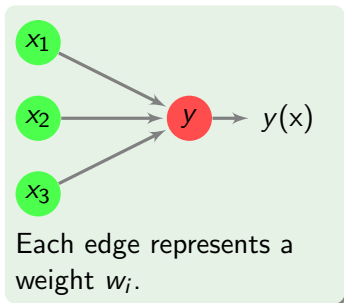


The prediction function is given by:

$$y(x) = \phi \left(b + \sum_i w_i x_i \right) \quad (5)$$

- ▶ if x is correctly classified ($y(x) \cdot t > 0$), the weights do not change.
- ▶ otherwise the weight vector is updated:
 $W \leftarrow W - \eta[y(x) - t] \cdot x$

Learning synaptic weights: Perceptron



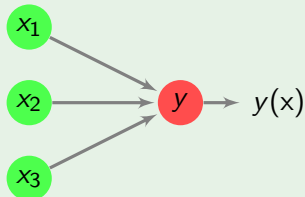
The prediction function is given by:

$$y(x) = \phi \left(b + \sum_i w_i x_i \right) \quad (5)$$

- ▶ if x is correctly classified ($y(x) \cdot t > 0$), the weights do not change.
- ▶ otherwise the weight vector is updated:

$$\begin{aligned} w_i &\leftarrow w_i - \eta[y(x) - t] \cdot x_i \\ b &\leftarrow b - \eta[y(x) - t] \end{aligned}$$

Learning synaptic weights: Gradient descent



Each edge represents a weight w_i .

The prediction function is given by:

$$\begin{aligned} y(x) &= \phi(z) \\ &= \phi\left(b + \sum_i w_i x_i\right) \end{aligned}$$

- ▶ We use the square loss function:

$$L(x, t) = \frac{1}{2}(y(x) - t)^2$$

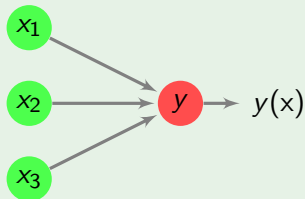
- ▶ Hence the derivative of the error is given by (chain rule):

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

- ▶ First term (derivative of the error):

$$\frac{\partial L}{\partial y} = \frac{\partial}{\partial y} \frac{1}{2}(y - t)^2 = y - t$$

Learning synaptic weights: Gradient descent



Each edge represents a weight w_i .

The prediction function is given by:

$$\begin{aligned} y(x) &= \phi(z) \\ &= \phi\left(b + \sum_i w_i x_i\right) \end{aligned}$$

- ▶ We use the square loss function:

$$L(x, t) = \frac{1}{2}(y(x) - t)^2$$

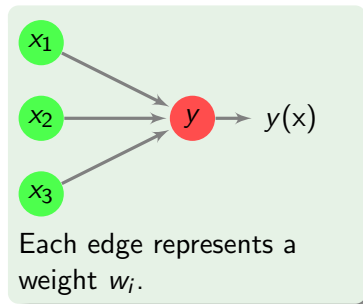
- ▶ Hence the derivative of the error is given by (chain rule):

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

- ▶ Second term (derivative of the nonlinearity):

$$\frac{\partial y}{\partial z} = \phi'(z)$$

Learning synaptic weights: Gradient descent



The prediction function is given by:

$$\begin{aligned} y(x) &= \phi(z) \\ &= \phi\left(b + \sum_i w_i x_i\right) \end{aligned}$$

- ▶ We use the square loss function:

$$L(x, t) = \frac{1}{2}(y(x) - t)^2$$

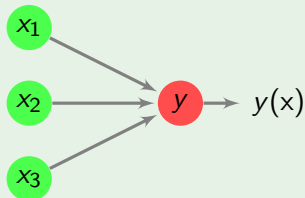
- ▶ Hence the derivative of the error is given by (chain rule):

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

- ▶ Third term (derivative of the linear part):

$$\frac{\partial z}{\partial w_i} = \frac{\partial}{\partial w_i} \left(b + \sum_k (w_k x_k) \right) = x_i$$

Learning synaptic weights: Gradient descent



Each edge represents a weight w_i .

The prediction function is given by:

$$\begin{aligned} y(x) &= \phi(z) \\ &= \phi\left(b + \sum_i w_i x_i\right) \end{aligned}$$

- ▶ We use the square loss function:

$$L(x, t) = \frac{1}{2}(y(x) - t)^2$$

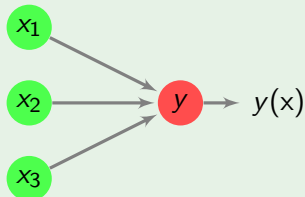
- ▶ Hence the derivative of the error is given by (chain rule):

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

- ▶ Overall, this comes to

$$\begin{aligned} \frac{\partial L}{\partial w_i} &= (y - t) \phi'(z) x_i \\ \frac{\partial L}{\partial b} &= (y - t) \phi'(z) \end{aligned}$$

Learning synaptic weights: Gradient descent



Each edge represents a weight w_i .

The prediction function is given by:

$$\begin{aligned} y(x) &= \phi(z) \\ &= \phi\left(b + \sum_i w_i x_i\right) \end{aligned}$$

- ▶ We use the square loss function:

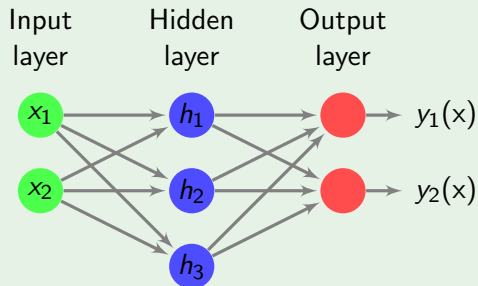
$$L(x, t) = \frac{1}{2}(y(x) - t)^2$$

- ▶ Hence the derivative of the error is given by (chain rule):

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

- ▶ $\delta = (y - t)\phi'(z)$
- ▶ $\frac{\partial L}{\partial w_i} = \delta x_i$
- ▶ the correction to the weight is called the **delta rule**: $\Delta w_i = -\eta \frac{\partial L}{\partial w_i}$ for a *learning rate* η .

Gradient descent in neural network

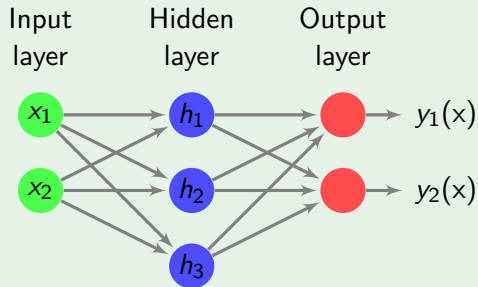


The parameters to learn are $\theta = (b^1, W^1, b^2, W^2)$.

The output units are given by

$$y_k(x) = \phi \left(b_k^2 + \sum_j W_{jk}^2 \phi \left(b_j^1 + \sum_i W_{ij}^1 x_i \right) \right) \quad (6)$$

Gradient descent in neural network



- ▶ We use the square loss function:
$$L(x, t) = \frac{1}{2}(t - y(x))^2$$
- ▶ Hence the derivative of the error is:

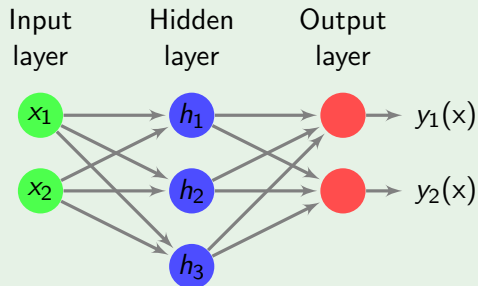
$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_{ij}}$$

For the **hidden layer** neurons, the loss is back-propagated from the output neurons. For a weight w_{ij} between input coming from previous hidden layer h_i and next layer hidden unit h_j :

$$\frac{\partial L_j}{\partial w_{ij}} = \delta_j h_i = \left(\sum_k \delta_k w_{jk}^2 \right) \phi'(z_j) h_i$$

See (link) for a full derivation.

Gradient descent in neural network



- ▶ We use the square loss function:
$$L(x, t) = \frac{1}{2}(t - y(x))^2$$
- ▶ Hence the derivative of the error is:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_{ij}}$$

$$\frac{\partial L_j}{\partial w_{ij}} = \delta_j h_i = \left(\sum_k \delta_k w_{jk}^2 \right) \phi'(z_j) h_i$$

The weights are updated using the same rule: $\Delta w_{ij} = -\eta \frac{\partial L_j}{\partial w_{ij}}$

Gradient descent in neural network

$$\frac{\partial L_j}{\partial w_{ij}} = \delta_j h_i = \left(\sum_k \delta_k w_{jk}^2 \right) \phi'(z_j) h_i$$

General rule

in order to calculate the weight gradients at any layer l in an arbitrarily-deep neural network, we simply need to calculate the backpropagated error signal δ that reaches that layer from the "above" layers, and weight it by the feed-forward signal h feeding into that layer.

Convergence, local minima and learning rate

- ▶ finding the ideal weights is a **non-convex problem**, and the solution space is likely to feature many local minima.
- ▶ gradient descent can get stuck in local minima, hence network weights are typically randomly optimised and the training may need to be restarted.
- ▶ The learning rate parameter η controls correction speed. Small η means small corrections, slow learning but low chances of missing a minimum; large η means large corrective steps and faster learning, but can make the network unstable.
- ▶ one suggestion is to use **momentum**

$$\Delta w_{ij}^{(n)} = \alpha \Delta w_{ij}^{(n-1)} - \eta \frac{\partial L_j}{\partial w_{ij}}$$

where the parameter $\alpha > 0$ is the momentum.

Stopping criterion

Question: When can we stop training the network?

Kramer & Sangiovanni-Vincentelli (1989)

“The back propagation algorithm is considered to have converged when the Euclidean norm of the gradient vector reaches a sufficiently small gradient threshold.”

Cross-validation

Evaluate network error on a separate training set and stop training when it stops improving.

Stochastic gradient descent and mini-batches

- ▶ In practice, it is very inefficient to correct weights for every training example.
- ▶ It is customary to use **mini-batches**, where the gradient is the average of the gradients inside a block B of $|B|$ examples.

$$\Delta w_{ij} = -\eta \frac{1}{|B|} \sum_{x \in B} \frac{\partial L(x)}{\partial w_{ij}}$$

- ▶ if $B = 1$, we have the usual stochastic gradient, with $B = N$ we have the batch gradient descent.

- 1 Artificial neural networks
 - Artificial Neuron: The Perceptron
 - Learning synaptic weights: Gradient descent
- 2 Convolutional Neural Networks
 - LeNet and AlexNet
 - VGG16 and VGG19
 - Generalisation and transfer learning
 - Deep Residual Networks
- 3 Adversarial analysis

Deep networks: The vanishing gradient problem

- ▶ In the previous slides, we have seen a three layers neural networks that works well for a variety of classification problems. Can we extend this by adding more hidden layers?
- ▶ Unfortunately, adding hidden layers does not help (and sometimes hurts!) performance.
- ▶ Recall that the generic weight gradient is given by

$$\frac{\partial L}{\partial w_{ij}^l} = \delta_i^l h_i^{l-1} = \left(\sum_k \delta_k^{l+1} w_{jk}^{l+1} \right) \phi'(z_j^l) h_i^{l-1}$$

Assuming a trivial network with 5 layers and only 1 neuron per layer, we have:

$$\frac{\partial L}{\partial w^5} = [t - y(x)]\phi'(z^5)h^4 \quad (7)$$

$$\frac{\partial L}{\partial w^4} = [t - y(x)]\phi'(z^5)w^5\phi'(z^4)h^3 \quad (8)$$

$$\frac{\partial L}{\partial w^3} = [t - y(x)]w^5w^4\phi'(z^5)\phi'(z^4)\phi'(z^3)h^2 \quad (9)$$

$$\frac{\partial L}{\partial w^2} = [t - y(x)]w^5w^4w^3\phi'(z^5)\phi'(z^4)\phi'(z^3)\phi'(z^2)h^1 \quad (10)$$

$$\frac{\partial L}{\partial w^1} = [t - y(x)]w^5w^4w^3w^2\phi'(z^5)\phi'(z^4)\phi'(z^3)\phi'(z^2)\phi'(z^1)x \quad (11)$$

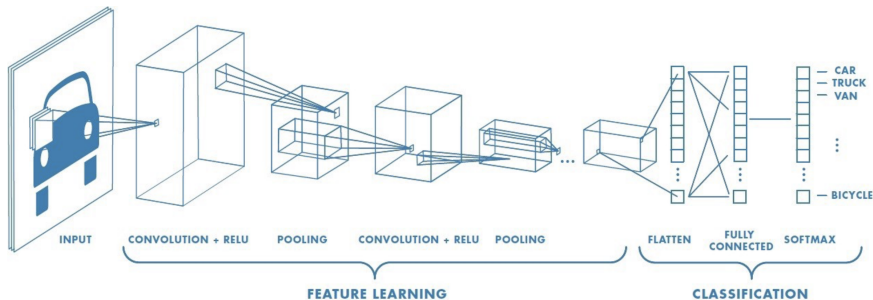
Clearly, if weights are smaller than 1, the gradient diminishes with each additional layer, hence lower layers train more slowly. This is called the **vanishing gradient problem**.

Limitations with standard neural networks

- ▶ When processing large number of inputs (eg, images) many hidden variables are likely to be needed, and the large number of parameters increases drastically the training data required.
- ▶ Such networks have no mechanism to encode translational invariance for temporal or spatial signals.
- ▶ Finally, the topology of the input is discarded: pixels in an image are not arranged at random!

- ▶ A convolutional neural network is a sparsely connected network with a special architecture, where each image location is processed with the same weights.
- ▶ Each neuron n at a convolutional layer generates a feature map F_n , where each pixel encodes this feature response at this image location.
- ▶ Convolutional layers are alternated with subsampling layers.
- ▶ Hence, each subsequent convolutional layer encodes more abstract feature that cover larger areas of the image.
- ▶ Classification is typically done by adding a fully connected neural network on the last layer's feature map.

Typical convolutional neural network architecture



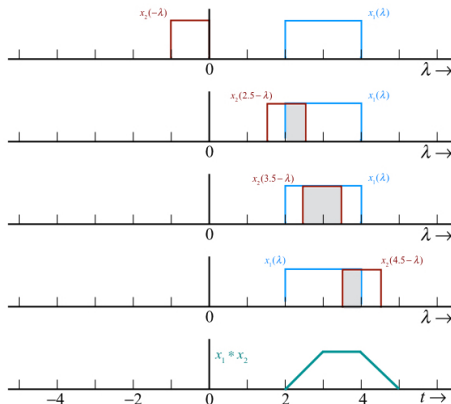
- ▶ Deep architectures alternate feature layers and subsampling
- ▶ **Feature layers** are convolutional: weights are shared at each image location.
- ▶ **Subsampling layers** (e.g. pooling) implement some translation distortion robustness in high level features.
- ▶ The final layer is a fully connected neural network.

Convolution

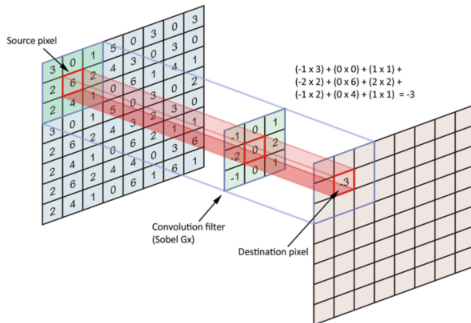
A convolution between two functions is given by:

$$(f \star g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(t - \tau)d\tau$$

Example:



Convolution layers



The feature map h^k is obtained by the following operation:

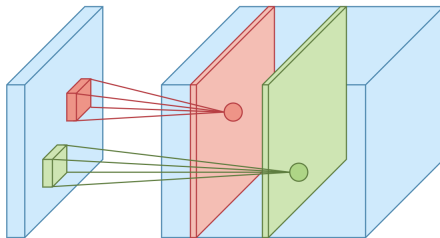
$$h_{ij}^k = \phi \left((W^k \star x)_{ij} + b_k \right)$$

where $(f \star g)_{ij}$ indicate the convolution operation:

$$(f \star g)_{ij} = \sum_{u=-\infty}^{+\infty} \sum_{v=-\infty}^{+\infty} f(u, v) g(i - u, j - v)$$

Convolution

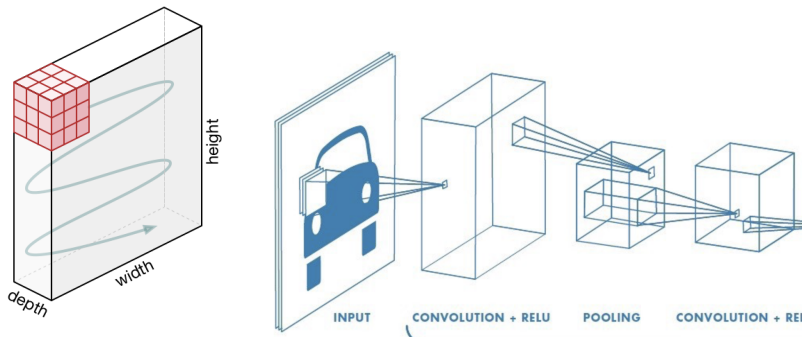
The convolutions are performed independently and the resulting feature maps are typically stacked along the depth dimension.



Convolution

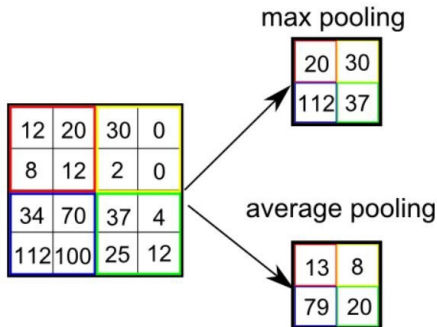
The convolutions operate on tensors in input, i.e. work on multiple channels.

E.g. input images generally have a depth of 3 (i.e. 3 color channels), while intermediate layers have depths in the 100s.



Subsampling layers: Pooling

- ▶ Pooling layers downsample each feature map independently, reducing the height and width, keeping the depth intact.
- ▶ By reducing dimensionality, it also reduces computational cost for downstream layers.
- ▶ It provides *some* translation invariance: if a feature is shifted by a small distance (ie, less than the pooling range), the response stays the same.



Subsampling layers: stride

- ▶ When the **stride** (i.e. the shift between successive applications) is smaller than the size of pooling, then the subsampling units **overlap**.

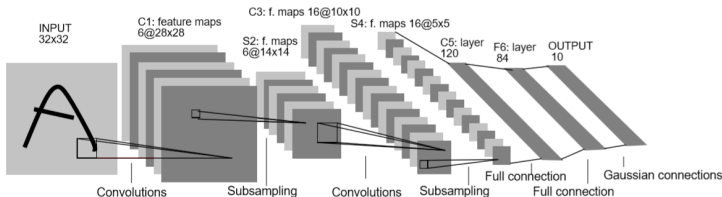
3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Digits classification: LeNet (1998)

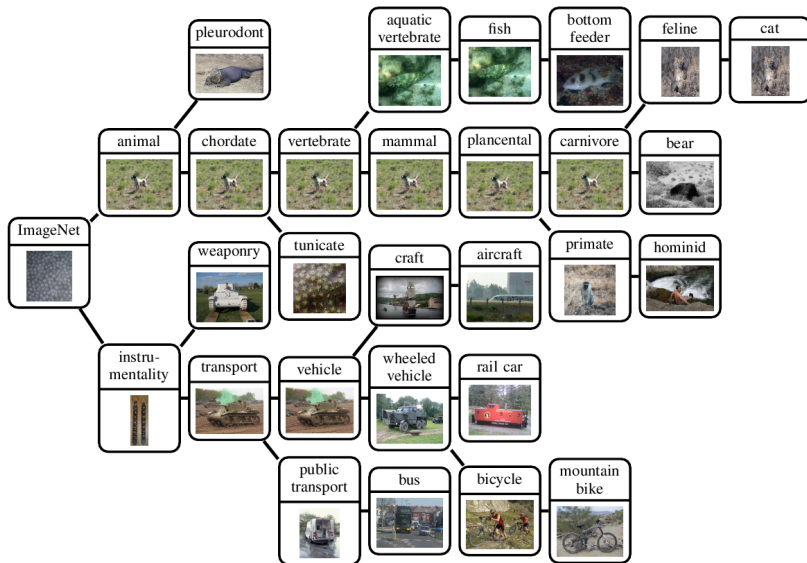


- ▶ Proposed by LeCun, Bottou, Bengio and Haffner in 1998 for solving handwritten digit recognition.
- ▶ 60,000 training examples, 10,000 test samples.
- ▶ reached 0.95% error rate vs. 4.7% for a 2-layer NN (300 hid. units).

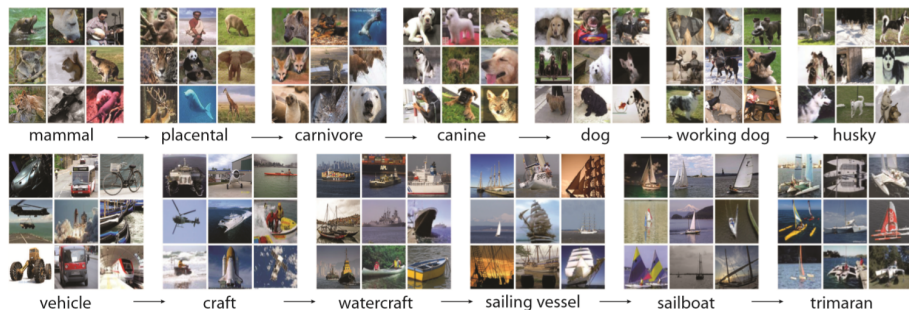
3 6 8 1 7 9 6 6 9 1
6 7 5 7 8 6 3 4 8 5
2 1 7 9 7 1 2 8 4 5

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. "Gradient-based learning applied to document recognition". *Proceedings of the IEEE* 86(11):2278–2324.

ImageNet classification: AlexNet (2012)

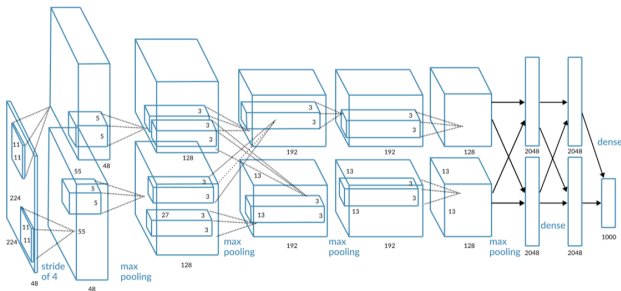


ImageNet classification: AlexNet (2012)



- ▶ Subset of ImageNet
- ▶ Contains 1,000 object classes, and about 1,000 examples per class.
- ▶ in total: 1.2 million training images, 50,000 validation and 150,000 testing images.

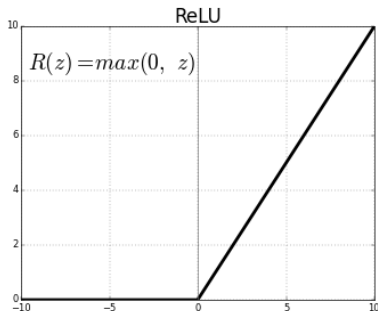
ImageNet classification: AlexNet (2012)



- ▶ 8 layers architecture: 5 convolutional + 3 fully connected.
- ▶ last layer is connected to a 1,000-way softmax for 1,000 classes.
- ▶ optimised to run on two GPUs
- ▶ batch B of 128 samples, momentum of 0.9 and weight decay of 0.0005; update rule:

$$\Delta w_i^{(n)} = 0.9 \Delta w_i^{(n-1)} - 0.0005 \eta w_i - \eta \frac{1}{|B|} \sum_{x \in B} \frac{\partial L}{\partial w_i}$$

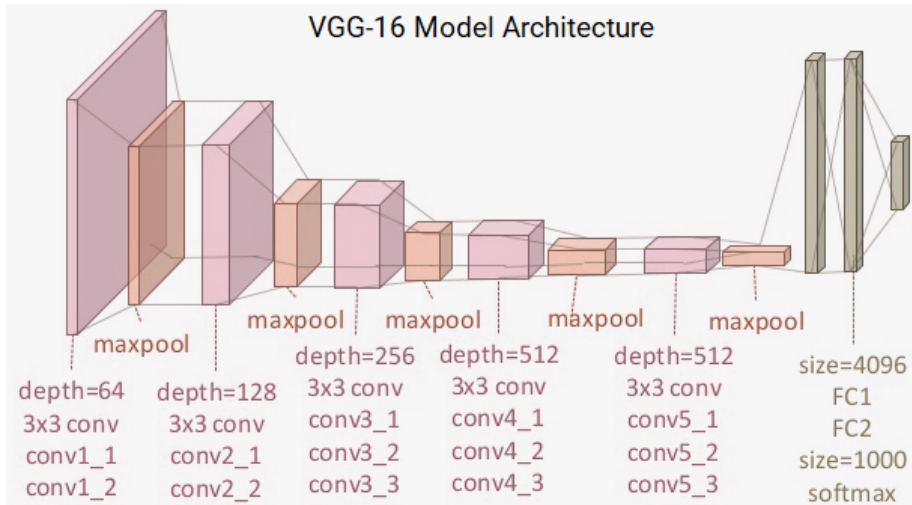
ReLU non-linearity



$$\phi(z) = \max(0, z)$$

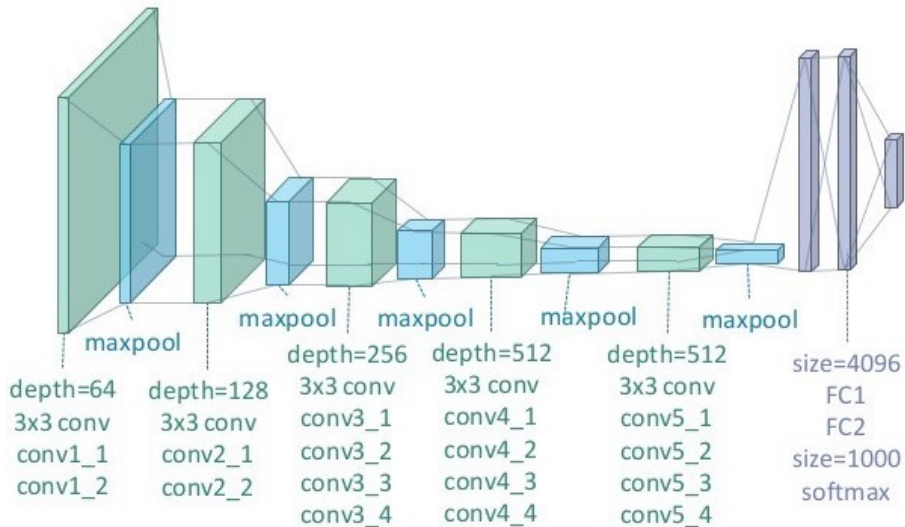
- ▶ AlexNet popularized the **Rectified Linear Units** as non-linearity of choice for CNNs.
- ▶ it leads to significantly better performance in ImageNet (result has since held on other applications of CNN) compared to classical tanh.

Stacking layers: VGG16



K. Simonyan, A. Zisserman (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition". *International Conference on Learning Representations*.

Stacking layers: VGG19



K. Simonyan, A. Zisserman (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition". *International Conference on Learning Representations*.

Stacking layers: VGG19

Model	top-5 classification error (ILSVRC-2012)	
	validation	test
VGG16	7.5%	7.4%
VGG19	7.5%	7.3%

K. Simonyan, A. Zisserman (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition". *International Conference on Learning Representations*.

- ▶ Both models yield very high performance!
- ▶ No significant difference between them
- ▶ Both models are available to download on most libraries.
- ▶ VGG19 is larger, require more GPU memory, but can perform better on some tasks.

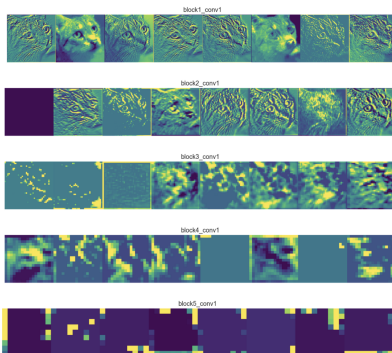
Question

We have evidence that CNNs can provide high performance on classification tasks. But are the features learnt by the hierarchy generic or class specific?

- ▶ There is increasing evidence that the extracted features are generic!
- ▶ In 2014, Sermanet and colleagues showed that a CNN trained on ImageNet could perform well on another dataset with only an additional fine tuning stage.
- ▶ Since, using a network **pre-trained** on ImageNet has become commonplace.
- ▶ CNNs have provided state-of-the-art (and sometimes human-level) performance for a variety of tasks: object recognition, object detection, face recognition.

Convolutional feature maps

- ▶ Initial convolutions act as edge detectors.
- ▶ Later convolutions encode abstract concepts and do not look like interpretable images any more
- ▶ Later convolutions are more sparse as the high level abstract concepts can be entirely missing from a specific image



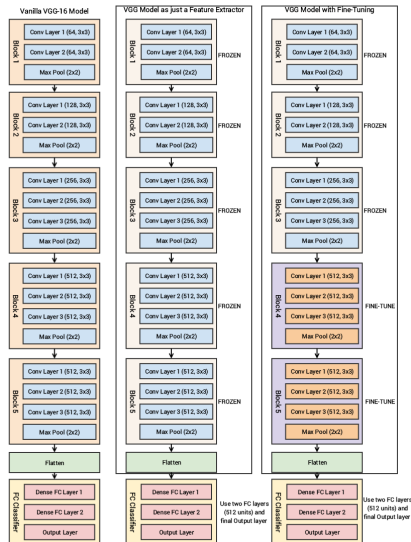
Pre-trained models

Idea: If features extracted by a Deep Neural Network (DNN) are generic to the feature space and not specific to the task, we can re-use them to train quickly a model for a different task.

- ▶ In practice we download a DNN pre-trained on a large dataset (ImageNet), remove the last fully connected layers that perform the actual classification and replace them with new ones for the new task
- ▶ We can then re-train those layers faster, with less data.
- ▶ The DNN performs as a **feature extractor**

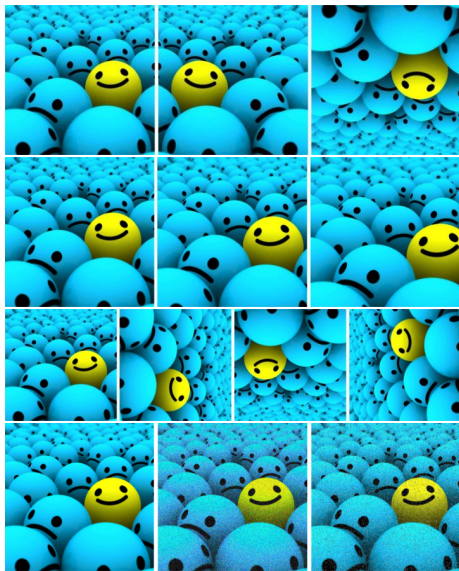
Pre-trained models & fine tuning

- 1 Download a DNN **pre-trained** on a large dataset (ImageNet)
- 2 Discard the last fully connected layers.
- 3 Either:
 - ▶ retrain new fully connected layers on the new data, **freezing** all convolutional layers
 - ▶ or **fine-tune** the convolutional layers in addition to training the new FC layers.



Data augmentation

- ▶ CNNs are robust to translations but not to other transformations.
- ▶ they can learn a robustness to other transformation, if enough training data is provided.
- ▶ one successful approach is to perform **data augmentation**:
- ▶ training images are transformed typically by taking a crop of a part of the image, and/or flipping the image around the y-axis.



<https://nanonets.com/blog/>

Dropout

- **Idea:** Force the network to learn a more robust representation by injecting noise during learning.
- Consider a neural network layer output $\mathbf{h}$, then dropout is defined as

$$\mathbf{h}_{\text{drop}} = (\mathbf{1}_{|h|} + \boldsymbol{\eta}_q) \odot \mathbf{h}$$

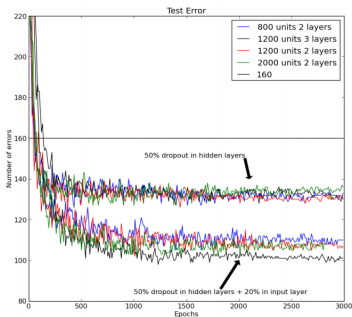
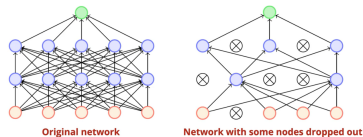
for $\boldsymbol{\eta}_q$ a perturbation with probability q :

$$(\boldsymbol{\eta}_q)_k = \begin{cases} -1 & \text{with probability } q \\ \frac{q}{q-1} & \text{with probability } 1 - q \end{cases}$$

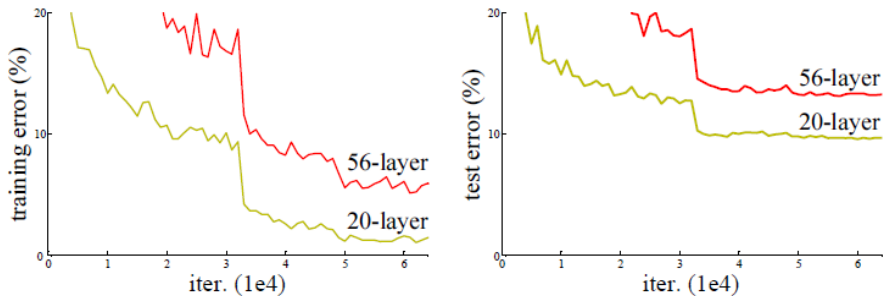
for each output k of $\mathbf{h}$

- This is a form of **implicit regularisation**.

Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. R. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580, 2012.



What about very deep networks



(results on the CIFAR-10 dataset <https://www.cs.toronto.edu/~kriz/cifar.html>)

Figure reproduced from He, Zhang, Ren and Sun (2016).

- ▶ Experiments showed that performance starts decreasing beyond a certain depth: this is the *degradation* problem.
- ▶ More surprisingly, even the *training* error starts increasing, when the depth goes beyond ~ 20 .

He, Zhang, Ren and Sun (2016). "Deep Residual Learning for Image Recognition." In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

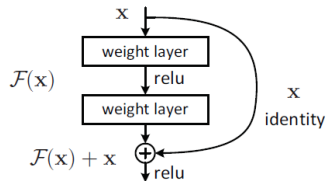
Deep residual networks

Idea

Consider a *residual* mapping such that:

$$\mathcal{H}(x) = \mathcal{F}(x) + x$$

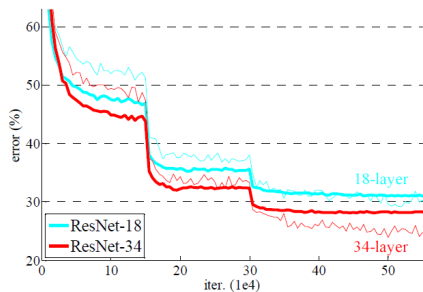
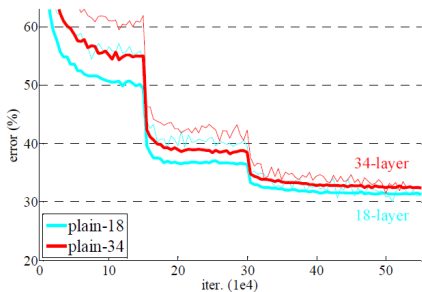
“We hypothesize that it is easier to optimize the residual mapping $\mathcal{H}(x)$ than to optimize the original, unreferenced mapping $\mathcal{F}(x)$.”
(He et al, 2016)



- Illustration of three deep architectures:
 - Left: classic VGG19 architecture.
 - middle: plain 34-layers architecture.
 - right: 34-layers residual architecture.



Comparison on ImageNet



Outline

- 1 Artificial neural networks
 - Artificial Neuron: The Perceptron
 - Learning synaptic weights: Gradient descent
- 2 Convolutional Neural Networks
 - LeNet and AlexNet
 - VGG16 and VGG19
 - Generalisation and transfer learning
 - Deep Residual Networks
- 3 Adversarial analysis

What do CNNs really learn?

- ▶ We have discussed theoretical reasons why hierarchical approaches should be successful at pattern recognition tasks.
- ▶ Also, we have argued from neuroscience evidence that the human visual system is likely using such a hierarchical strategy.
- ▶ CNNs in particular appear to reach human-like pattern recognition performance.

Question

Are the feature hierarchies learnt by CNNs similar to the ones used by the human brain?

What do CNNs really learn?

- ▶ We have discussed theoretical reasons why hierarchical approaches should be successful at pattern recognition tasks.
- ▶ Also, we have argued from neuroscience evidence that the human visual system is likely using such a hierarchical strategy.
- ▶ CNNs in particular appear to reach human-like pattern recognition performance.

Question

Are the feature hierarchies learnt by CNNs similar to the ones used by the human brain?

- ▶ At the lower layers, this seems to be the case, receptive fields of the neurons are similar to the cortical V1 area.
- ▶ But what of the other layers?
- ▶ unfortunately, analysing artificial neural networks is hard.

Learning minimal perturbations

Question

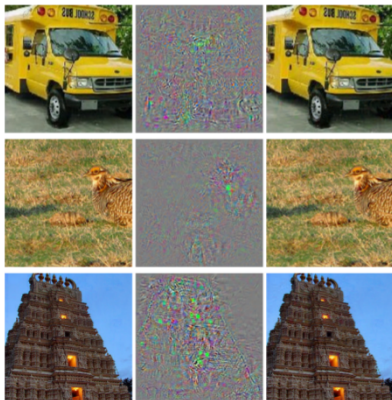
Can we find the most similar input patterns that lead to a misclassification? (ie, can we test the margin of our classifier?)

- ▶ Let's consider a classifier $f : \mathbb{R}^m \rightarrow \{1 \dots C\}$
- ▶ Assume we also have a loss function $L_f : \mathbb{R}^m \times \{1 \dots C\} \rightarrow \mathbb{R}^+$
- ▶ We solve the following optimisation problem, for a given class $\lambda \in \{1 \dots C\}$:

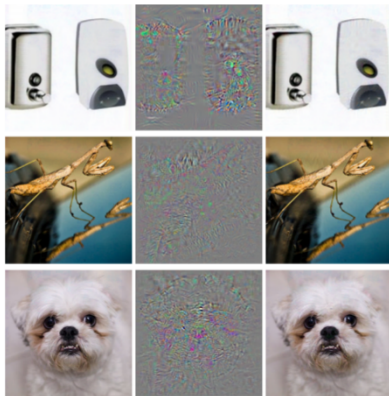
minimise $\|\mathbf{r}\|_2$ subject to:

- ▶ $f(\mathbf{x} + \mathbf{r}) = \lambda$
- ▶ $\mathbf{x} + \mathbf{r} \in [0, 1]^m$
- ▶ This is hard to optimise, so in practice we approximate it with:
arg min $_{\mathbf{c}, \mathbf{r}} \mathbf{c}|\mathbf{r}| + L_f(\mathbf{x} + \mathbf{r}, \lambda)$
subject to $\mathbf{x} + \mathbf{r} \in [0, 1]^m$ and $\mathbf{c} > 0$.

Joan Bruna, Christian Szegedy, Ilya Sutskever, Ian Goodfellow, Wojciech Zaremba, Rob Fergus, Dumitru Erhan (2014). "Intriguing properties of neural networks." *ICLR'2014*.

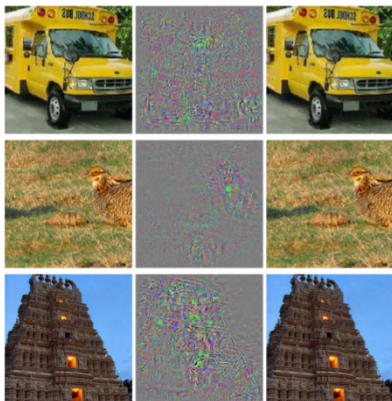


(a)



(b)

What is detected on the right columns?



(a)



(b)

Ostriches!

Universal perturbations

This goes further. Recently another team showed that universal perturbations could be defined.

- ▶ This means we seek an perturbation vector $r \in \mathbb{R}^m$ such that

$$f(x + r) \neq f(x)$$

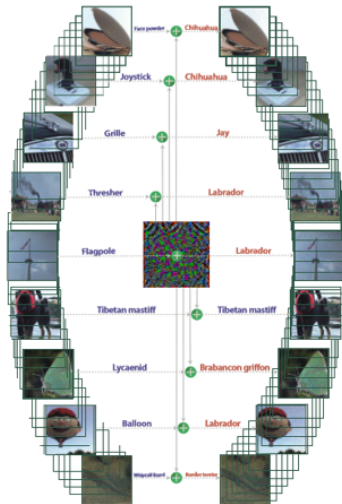
for **most** $x \sim \mu$

- ▶ This is called a “universal” perturbation.
- ▶ In practice, we want to find r that satisfies:
 - ▶ $\|r\|_p \leq \xi$
 - ▶ $p[f(x + r) \neq f(x)] \geq 1 - \delta$

Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, Pascal Frossard (2017)
“Universal adversarial perturbations” *IEEE Conference on Computer Vision and Pattern Recognition*.

Universal perturbations: Examples

- ▶ This approach can generate patterns that lead to fooling rates of 80 to 90%
- ▶ interestingly, a universal perturbation trained on one CNN architecture can still achieve high fooling rate on another network architecture:
- ▶ Eg, perturbation trained on GoogLeNet to achieve a fooling rate of 78%, still achieves a fooling rate of 40% on VGG-16.



- ▶ CNNs are highly effective approaches for pattern recognition, achieving performance beyond any pre-existing approach.
- ▶ Using pre-trained networks allows effective use on a variety of problems.
- ▶ Use of parallel processing on GPU allows fairly efficient processing.
- ▶ but...
- ▶ Universal perturbations experiments show that CNNs, although competent at recognising patterns, achieve this performance through processes vastly different from human vision.
- ▶ This potential brittleness of CNNs is a potential concern for deployment (*adversarial hacking*)

Available packages

CNNs are expensive to train, so it is preferable to use GPU optimised code (and powerful GPUs):



TensorFlow: library maintained by Google <https://www.tensorflow.org/>



PyTorch: library maintained by Facebook <http://pytorch.org/>



MatConvNet for Matlab <http://www.vlfeat.org/matconvnet/>



CAFFE: ex-most popular codebase in computer vision research, maintained by Berkeley <http://caffe.berkeleyvision.org/>

Also, these libraries generally offer pre-trained models.